CORRIGÉ · PROBLÈME · ANNALES MP

Réseaux, chemins augmentants et plus petite bordure

Énoncé

Langage C (ou pseudo-code de même niveau). Un réseau $H = (W, F, \mu, s, t)$: sommets numérotés de 0 à $n+1$, source $s = 0$, puits $t = n+1$, multiplicité $\mu(e) \in \mathbb{N}^*$ pour chaque arc $e \in F$. Fichier `network.h` :

```
#ifndef NETWORK_H
#define NETWORK_H
const unsigned int NMAX = 102;
struct network_s { unsigned int n; unsigned mu[NMAX][NMAX]; };
typedef struct network_s network;
void nw_init(unsigned int n);
void nw_add(int u, int v);
void nw_remove(int u, int v);
void nw_disconnect(void);
void _arr_init(int *T);
bool _bfs_tree(void);
void _residue(void);
#endif
```

Fichier `network.c` : `network H; int A[NMAX];` ; `H.mu[u][v]` vaut $\mu((u,v))$, ou 0 si (u,v) n'est pas un arc.

1. Rôle des lignes `#ifndef`, `#define` et `#endif`.
2. Comment faire interagir `network.h` et `network.c` ?
3. `void nw_init(unsigned int n)` : précondition $n < n_{\max}$ vérifiée par une assertion ; `H` devient le réseau vide à $n+2$ sommets.
4. `void nw_add(int u, int v)` : incrémente $\mu[u][v]$.
5. `void nw_remove(int u, int v)` : retire une occurrence, avec une assertion si l'arc n'existe pas.
6. `void _arr_init(int *T)` : met les $n+2$ cases de `T` à -1.
7. `bool _bfs_tree(void)` : calcule dans `A` une arborescence de plus courts chemins (parcours en largeur depuis s) : `A[s] = s`, `A[v]` = prédécesseur pour v accessible, -1 sinon ; renvoie `true` si t est accessible.

Un ensemble Π de k chemins de s à t est réalisable si chaque arc e est utilisé au plus $\mu(e)$ fois. Le réseau résiduel $\mathrm{Res}(H, \Pi)$ s'obtient en retirant, pour chaque arc (u, v) de chaque chemin, une occurrence de (u, v) et en ajoutant une occurrence de (v, u).

1. `void _residue(void)` : remplace `H` par $\mathrm{Res}(H, \{\alpha\})$, où α est le chemin de s à t dans l'arborescence `A`.
2. Après k appels successifs (chemins $\alpha_1, \dots, \alpha_k$), montrer qu'il existe un ensemble réalisable Π de k chemins tel que le réseau obtenu soit $\mathrm{Res}(H, \Pi)$.
3. `void nw_disconnect(void)` : tant qu'il existe un chemin de s à t, remplacer H par le résiduel le long d'un plus court chemin ; à la fin, `A` est une arborescence de racine s n'atteignant pas t.
4. Montrer que `nw_disconnect` termine (variant inspiré de 9).

Bordure ∂S : arcs $(u, v) \in F$ avec $u \in S$, $v \notin S$; $\mu(\partial S) = \sum_{e \in \partial S} \mu(e)$. Un entourage est un $S \subseteq W$ avec $s \in S$, $t \notin S$.

1. Pour Π réalisable de k chemins et S entourage : $k \leqslant \mu(\partial S)$.

On fixe Π_0 réalisable de k_0 chemins tel que $\mathrm{Res}(H, \Pi_0)$ n'ait plus de chemin de s à t, et $S_0 =$ sommets accessibles depuis s dans $\mathrm{Res}(H, \Pi_0)$.

1. Pour tout arc (u, v) avec $u \in S_0$, $v \notin S_0$: toutes ses $\mu(e)$ occurrences sont utilisées par Π_0.
2. Aucune occurrence d'un arc (v, u) avec $v \notin S_0$, $u \in S_0$ n'est utilisée par Π_0.
3. Montrer $k_0 = \mu(\partial S_0)$.
4. Montrer $k_0 = \sigma(H)$ (plus petite bordure d'un entourage) et que, après `nw_disconnect`, $\{v : A[v] \geqslant 0\}$ est un entourage de plus petite bordure.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Les variables `H` et `A` sont globales : les fonctions n'ont presque pas d'arguments.
- Multiplicité = nombre de copies d'un arc ; résiduel = on « retourne » une copie de chaque arc emprunté.

Ce qu'on te demande. Construire des chemins réalisables jusqu'à déconnecter s de t, puis prouver que leur nombre est la plus petite bordure (forme du théorème flot max / coupe min).

Corrigé

Question 1

Étape 1. Q1. Rôle de la garde d'inclusion

À la première inclusion, `NETWORK_H` n'est pas défini : `#ifndef NETWORK_H` laisse passer le contenu, et `#define NETWORK_H` le définit. À toute inclusion suivante dans le même fichier compilé, `#ifndef` est faux et le préprocesseur saute tout jusqu'au `#endif`. Cela évite les doubles définitions (de `struct network_s`, du `typedef`…), qui provoqueraient des erreurs de compilation.

Question 2

Étape 1. Q2. Faire interagir les deux fichiers

- `network.c` commence par `#include "network.h"` (guillemets : fichier local), qui lui donne la structure, la constante et les prototypes.
- Tout autre fichier qui utilise le réseau inclut aussi `network.h`.
- On compile séparément (`gcc -c network.c` donne `network.o`), puis on lie les objets : `gcc network.o main.o -o prog`.

Remarques de rigueur : en C (contrairement au C++), `const unsigned int NMAX` n'est pas une expression constante, donc ne peut pas dimensionner un tableau dans une `struct`, et une définition de variable dans un en-tête inclus par plusieurs fichiers provoque une double définition à l'édition de liens. On écrirait plutôt `#define NMAX 102`. De même, il faut `#include <stdbool.h>` pour le type `bool`.

Question 3

Étape 1. Q3. nw_init

```
void nw_init(unsigned int n) {
    assert(n < NMAX);              /* précondition (voir remarque) */
    H.n = n;
    for (unsigned int u = 0; u < n + 2; u++)
        for (unsigned int v = 0; v < n + 2; v++)
            H.mu[u][v] = 0;        /* aucun arc */
}
```

Remarque : les indices vont jusqu'à $n+1$; pour ne pas déborder, il faudrait en fait $n+2 \leqslant n_{\max}$ (soit `assert(n + 2 <= NMAX)`). La précondition de l'énoncé, $n < n_{\max}$, est un peu trop faible.

Question 4

Étape 1. Q4. nw_add

```
void nw_add(int u, int v) {
    H.mu[u][v] = H.mu[u][v] + 1;   /* une occurrence de plus de l'arc (u,v) */
}
```

Question 5

Étape 1. Q5. nw_remove

```
void nw_remove(int u, int v) {
    assert(H.mu[u][v] > 0);        /* l'arc doit exister */
    H.mu[u][v] = H.mu[u][v] - 1;
}
```

L'assertion est indispensable : `mu` est non signé, $0 - 1$ donnerait un très grand entier.

Question 6

Étape 1. Q6. _arr_init

```
void _arr_init(int *T) {
    for (unsigned int v = 0; v < H.n + 2; v++)
        T[v] = -1;
}
```

Question 7

Étape 1. Q7. Parcours en largeur

```
bool _bfs_tree(void) {
    int N = H.n + 2;               /* nombre de sommets */
    int file[NMAX];                /* chaque sommet est enfilé au plus une fois */
    int debut = 0, fin = 0;
    _arr_init(A);
    A[0] = 0;                      /* la source est sa propre racine */
    file[fin++] = 0;
    while (debut < fin) {
        int u = file[debut++];     /* on défile */
        for (int v = 0; v < N; v++)
            if (H.mu[u][v] > 0 && A[v] == -1) {   /* arc (u,v), v pas encore vu */
                A[v] = u;
                file[fin++] = v;
            }
    }
    return A[N - 1] != -1;         /* le puits t = n+1 est-il atteint ? */
}
```

Un sommet reçoit son prédécesseur à sa découverte et n'est enfilé qu'une fois, donc la file de taille $n+2$ suffit. Le parcours en largeur découvre les sommets par distance croissante : l'arborescence donne des plus courts chemins. Complexité $O(n^2)$ (matrice d'adjacence).

Question 8

Étape 1. Q8. _residue

```
void _residue(void) {
    int v = H.n + 1;               /* on part du puits */
    while (v != 0) {               /* jusqu'à la source */
        int u = A[v];              /* arc (u,v) du chemin alpha */
        nw_remove(u, v);           /* une occurrence de (u,v) en moins */
        nw_add(v, u);              /* une occurrence de l'arc opposé en plus */
        v = u;
    }
}
```

On parcourt le chemin α à l'envers ; chaque arc est traité une fois, comme dans la définition du réseau résiduel.

Question 9

Étape 1. Q9. Des résiduels successifs à un seul résiduel

Idée : on enregistre, pour chaque arc $e = (u, v) \in F$, un entier $f(e)$ (combien de fois e est utilisé « net »), tel que le réseau courant soit

$$\mu_{\text{cour}}(u, v) = \mu(u, v) - f(u, v) + f(v, u)$$

(on retire les copies utilisées et on ajoute les copies retournées). Au départ $f = 0$.

Mise à jour quand on applique `_residue` avec un chemin α : pour chaque arc (u, v) de α, si $f(v, u) > 0$ on fait $f(v, u) \leftarrow f(v, u) - 1$ (on annule un passage en sens inverse), sinon $f(u, v) \leftarrow f(u, v) + 1$. Dans les deux cas, $\mu_{\text{cour}}(u, v)$ baisse de 1 et $\mu_{\text{cour}}(v, u)$ monte de 1 : c'est exactement l'effet de `_residue`. Comme (u, v) existe dans le réseau courant, on garde $0 \leqslant f \leqslant \mu$.

Invariant après j appels : $0 \leqslant f \leqslant \mu$; en tout sommet autre que s et t, autant d'unités de f entrent que sortent (un chemin qui entre dans un sommet en ressort) ; il sort j unités nettes de s.

Décomposition : on extrait un chemin de s à t formé d'arcs où $f > 0$ (en partant de s et en suivant des arcs « chargés », la conservation garantit qu'on ne reste pas bloqué avant t) ; on enlève 1 sur ses arcs et on recommence. On obtient k chemins Π qui utilisent chaque arc e exactement $f(e) \leqslant \mu(e)$ fois : Π est réalisable et $\text{Res}(H, \Pi)$ a pour multiplicités $\mu - f(u, v) + f(v, u)$, c'est-à-dire le réseau obtenu.

(S'il reste après extraction des circuits d'unités de flot sans lien avec s, on les traite comme des passages supplémentaires sur des arcs déjà comptés ; ils ne changent ni k ni les raisonnements qui suivent, qui ne portent que sur les arcs qui sortent ou entrent dans un entourage.)

Question 10

Étape 1. Q10. nw_disconnect

```
void nw_disconnect(void) {
    while (_bfs_tree())           /* tant que t est accessible depuis s */
        _residue();               /* résiduel le long d'un plus court chemin */
}
```

Le dernier appel à `_bfs_tree` a renvoyé `false` : `A` contient alors une arborescence de racine s dans le réseau résiduel final, qui n'atteint pas t, comme demandé.

Question 11

Étape 1. Q11. Terminaison

Après k tours, par Q9, le réseau est $\text{Res}(H, \Pi)$ pour un ensemble réalisable Π de k chemins. Par Q12 appliquée à l'entourage $S = \{s\}$ (qui contient s et pas t) :

$$k \leqslant \mu(\partial\{s\}) = \sum_{v} \mu(s, v) \qquad \text{(multiplicités du réseau de départ).}$$

Variant : $\mu(\partial\{s\}) - k$ est un entier naturel qui diminue de 1 à chaque tour. La boucle fait donc au plus $\mu(\partial\{s\})$ tours : elle termine.

Question 12

Étape 1. Q12. Chaque chemin traverse la bordure

Chaque chemin π_j commence en $s \in S$ et finit en $t \notin S$: il contient au moins un arc qui sort de S, c'est-à-dire un arc de ∂S (le premier arc dont l'origine est dans S et l'extrémité hors de S). En comptant ces passages :

$$k \leqslant \sum_{e \in \partial S} \mathrm{card}\{j : e \in \pi_j\} \leqslant \sum_{e \in \partial S} \mu(e) = \mu(\partial S),$$

la seconde inégalité venant de la réalisabilité.

Question 13

Étape 1. Q13. Arcs sortants de S_0 saturés

Soit $e = (u, v) \in F$ avec $u \in S_0$, $v \notin S_0$. Dans $\mathrm{Res}(H, \Pi_0)$, la multiplicité de (u, v) vaut $\mu(e) -$ (utilisations de e) + (utilisations de (v, u)) $\geqslant \mu(e) -$ (utilisations de e).

Si une occurrence de e n'était pas utilisée, cette multiplicité serait $\geqslant 1$: l'arc (u, v) existerait dans le résiduel, et v serait accessible depuis s (puisque u l'est), donc $v \in S_0$: contradiction. Toutes les $\mu(e)$ occurrences sont donc utilisées.

Question 14

Étape 1. Q14. Aucun arc entrant dans S_0 n'est utilisé

Soit $(v, u) \in F$ avec $v \notin S_0$, $u \in S_0$. Dans le résiduel, la multiplicité de (u, v) vaut $\mu(u, v) -$ (utilisations de (u, v)) + (utilisations de (v, u)), avec $\mu(u, v) = 0$ si $(u, v) \notin F$.

Par Q13, si $(u, v) \in F$, toutes ses occurrences sont utilisées : les deux premiers termes se compensent. Il reste la multiplicité « utilisations de (v, u) ». Si elle était $\geqslant 1$, (u, v) serait un arc du résiduel et v serait accessible : contradiction. Donc (v, u) n'est jamais utilisé.

Question 15

Étape 1. Q15. $k_0 = \{\}mu(\{\}partialS_0)$

Un chemin de $s \in S_0$ à $t \notin S_0$ sort de S_0 une fois de plus qu'il n'y entre. D'après Q14, aucun chemin n'entre dans S_0 : chaque chemin sort donc de S_0 **exactement une fois**. Le nombre total d'utilisations d'arcs de ∂S_0 vaut donc k_0.

D'après Q13, ce même nombre vaut $\sum_{e \in \partial S_0} \mu(e) = \mu(\partial S_0)$ (toutes les occurrences sont utilisées). D'où

$$k_0 = \mu(\partial S_0).$$

Question 16

Étape 1. Q16. $k_0 = \{\}sigma(H)$ et entourage optimal

Par Q12, $k_0 \leqslant \mu(\partial S)$ pour tout entourage S, donc $k_0 \leqslant \sigma(H)$. Par Q15, $\sigma(H) \leqslant \mu(\partial S_0) = k_0$. D'où

$$k_0 = \sigma(H) = \mu(\partial S_0).$$

Après `nw_disconnect` : par Q9, le réseau obtenu est $\mathrm{Res}(H, \Pi_0)$ pour un Π_0 réalisable, et il n'a plus de chemin de s à t. Le dernier parcours en largeur a marqué ($A[v] \geqslant 0$) exactement les sommets accessibles depuis s, soit S_0. C'est un entourage (s marqué, t non) de multiplicité $\mu(\partial S_0) = \sigma(H)$: un entourage de plus petite bordure.

Le piège

En Q14, oublier qu'utiliser l'arc (v, u) ajoute une copie de (u, v) dans le résiduel : c'est elle qui rendrait v accessible.

La méthode

Ford-Fulkerson : augmenter tant qu'un chemin existe dans le résiduel ; à la fin, les sommets accessibles forment une coupe minimale (arcs sortants saturés, arcs entrants inutilisés).

Rappels de cours utilisés

- **Garde d'inclusion.** `#ifndef X` / `#define X` / `#endif` : le contenu n'est inclus qu'une fois par fichier compilé, même si l'en-tête est inclus plusieurs fois (directement ou par d'autres en-têtes).
- **Compilation séparée en C.** Chaque fichier `.c` inclut les en-têtes `.h` dont il a besoin et se compile en un fichier objet (`gcc -c f.c`). L'édition de liens assemble les objets (`gcc f.o g.o -o prog`). Une variable globale définie dans un fichier s'utilise ailleurs grâce à une déclaration `extern`.

- **Parcours en largeur.** File FIFO : on défile un sommet, on enfile ses voisins non encore vus en notant leur prédécesseur. Chaque sommet est enfilé au plus une fois ; l'arborescence obtenue donne des plus courts chemins en nombre d'arcs.
- **Coupe et chemins (Menger, Ford-Fulkerson).** Le nombre maximal de chemins réalisables de s à t est égal à la multiplicité minimale d'une bordure séparant s de t. Les chemins augmentants dans le réseau résiduel permettent de les construire.